FINAL PROJECT	
Course Code: CPE 009B	**Program:** Computer Engineering
Course Title: Object Oriented Programming 2	**Date Performed:** September 12, 2026
Section: CPE21S2	**Date Submitted:** October 24, 2026
Group Member(s): THOMAS JOSEPH ASTUDILLO JUAN PAULO LARA CARLO MENESES BENJ ARIEL RATCHO JOHREL LOUIE SILAO CHRISTIAN LEE BANDIGAN	**Instructor:** Engr. Noel Jason Lusung

A. Instruction

Simple Chat Application (Username, chat threading w/ emoji)

Tasks:

1. Optional (login: username/ pw, can access multiple accounts)
2. Username (avatar: optional)
3. Chat thread (date/time)
4. Limit text per chat 300char
5. Send Emoji
6. Compliment color palette (logo), send button, edit (optional)
7. Local (multiple users allowed)

Materials & Equipment

- Personal Computer (Windows 10/11)
- Python Version 3.13 – 3.14
- Visual Studio Code / Spyder IDE
- PyQt6

Discussion

Algorithm

- **Menu Interface**
  1. Load system menu

- **User Authentication**
  1. Check if the user wants to log in or switch accounts.
  2. If logging in. Prompt user for username and password.
  3. Check if the user's username and password is correct.
  4. If the user has successfully logged in. Proceed to the chat menu interface.

- **Chat Interface & Threads**
  1. Load existing chat messages.

- **Preparing and Validating Messages**
    1. Input message with or without emojis.
    2. Check character limit
    3. If the user's inputted text length is ≤ 300 characters. Can send messages.
    4. If the user's inputted text length is > 300, send out a warning prompt to alert the user.

- **Sending Messages**
    1. When clicking the send button:
        a. Prints out the date and time
        b. Saves the messages to a local file.
        c. Update the chat thread.

Procedures
Code
Menu Interface

Python

```
import sys
from PyQt6.QtCore import Qt, QByteArray, QSize, QRectF, QPointF, QPropertyAnimation, QPoint
from PyQt6.QtWidgets import (
    QApplication, QWidget, QGridLayout, QHBoxLayout, QLabel, QLineEdit, QPushButton, QMessageBox
)
from PyQt6.QtGui import QPixmap, QPainter, QPainterPath, QIcon, QColor, QFont
from PyQt6.QtSvg import QSvgRenderer

from AccountRegistration import RegistrationApp
from database import create_tables, get_user, get_all_users

BG_COLOR = "#F5F5DC"
INK_COLOR = "#000000"
BTN_COLOR = "#57534B"
BTN_HOVER = "#3F3C36"
BTN_TEXT = "#F5F5DC"
LOGO_BUBBLE = "#D5D1BB"
FACE_FILL = "#E8CBAE"
FACE_LINE = "#4A2A1A"

ICON_USERNAME = (
    '<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24">'
    '<g fill="none" stroke="#000" stroke-width="1.8" stroke-linecap="round">'
    '<rect x="2.5" y="4" width="19" height="16" rx="3"/>'
```

```
    '<circle cx="8.5" cy="10" r="2"/><path d="M5.5 16c.5-2 2-2.8 3-2.8s2.5.8 3 2.8"/>'
    '<path d="M14.5 10h4M14.5 14h4"/></g></svg>'
)

ICON_LOCK = (
    '<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" fill="#000">'
    '<path d="M18 8h-1V6c0-2.76-2.24-5-5-5S7 3.24 7 6v2H6c-1.1 0-2 .9-2 2v10c0 1.1.9 2 2 2h12'
    'c1.1 0 2-.9 2-2V10c0-1.1-.9-2-2-2zm-6 9c-1.1 0-2-.9-2-2s.9-2 2-2 2 .9 2 2-.9 2-2 2zm3.1-9'
    'H8.9V6c0-1.71 1.39-3.1 3.1-3.1 1.71 0 3.1 1.39 3.1 3.1v2z"/></svg>'
)

FACE_BODY = (
    f'<path d="M24 5C12 5 3 12 3 22c0 5 2.5 9 6.5 12L6 43l9-5c3 1.2 5.5 1.5 9 1.5 '
    f'12 0 21-7 21-17.5S36 5 24 5z" fill="{FACE_FILL}" stroke="{FACE_LINE}" '
    f'stroke-width="2.4" stroke-linejoin="round"/>'
    f'<path d="M12 21q4-3.5 8 0M28 21q4-3.5 8 0" fill="none" stroke="{FACE_LINE}" '
    f'stroke-width="1.8" stroke-linecap="round"/>'
)

FACE_SVG = f'<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 48 48">{FACE_BODY}</svg>'

APP_ICON_SVG = (
    '<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64">'
    f'<rect width="64" height="64" rx="14" fill="{LOGO_BUBBLE}"/>'
    f'<g transform="translate(8 8)">{FACE_BODY}</g></svg>'
)


def svg_to_pixmap(svg_text, size):
    renderer = QSvgRenderer(QByteArray(svg_text.encode("utf-8")))
    pixmap = QPixmap(size, size)
    pixmap.fill(Qt.GlobalColor.transparent)
    painter = QPainter(pixmap)
    renderer.render(painter)
    painter.end()
    return pixmap


def make_app_icon():
    icon = QIcon()
    for s in (16, 32, 48, 64, 128, 256):
        icon.addPixmap(svg_to_pixmap(APP_ICON_SVG, s))
    return icon


class LogoWidget(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)
```

```
        self.setFixedSize(262, 134)
        self.face = QSvgRenderer(QByteArray(FACE_SVG.encode("utf-8")))

    def paintEvent(self, event):
        painter = QPainter(self)
        painter.setRenderHint(QPainter.RenderHint.Antialiasing)

        bubble = QPainterPath()
        bubble.addRoundedRect(QRectF(4, 4, 254, 97), 30, 30)

        tail = QPainterPath()
        tail.moveTo(QPointF(74, 96))
        tail.lineTo(QPointF(64, 128))
        tail.lineTo(QPointF(112, 96))
        tail.closeSubpath()

        painter.setPen(Qt.PenStyle.NoPen)
        painter.setBrush(QColor(LOGO_BUBBLE))
        painter.drawPath(bubble.united(tail))

        self.face.render(painter, QRectF(24, 26, 48, 48))

        font = QFont("Segoe UI")
        font.setPixelSize(38)
        painter.setFont(font)
        painter.setPen(QColor(INK_COLOR))
        painter.drawText(QRectF(82, 4, 170, 97), int(Qt.AlignmentFlag.AlignVCenter | Qt.AlignmentFlag.AlignLeft),
"DChat")
        painter.end()


class App(QWidget):
    def __init__(self):
        super().__init__()
        self.title = "Menu & Login Screen"
        self.initUI()

    def initUI(self):
        self.setWindowTitle(self.title)
        self.setWindowIcon(make_app_icon())
        self.setGeometry(200, 200, 560, 400)
        self.setMinimumSize(480, 380)

        self.setStyleSheet(f"""
            QWidget {{
                background-color: {BG_COLOR};
                color: {INK_COLOR};
                font-family: 'Segoe UI', sans-serif;
```

```
        }}
        QLabel {{ font-size: 16px; background: transparent; }}
        QLineEdit {{
            background-color: transparent;
            color: {INK_COLOR};
            border: 2px solid {INK_COLOR};
            border-radius: 19px;
            padding: 0 14px;
            font-size: 14px;
            min-width: 200px;
            min-height: 34px;
        }}
        QLineEdit:focus {{ background-color: #FFFFFF; }}
        QPushButton#loginBtn, QPushButton#regBtn {{
            background-color: {BTN_COLOR};
            color: {BTN_TEXT};
            border: 2px solid {INK_COLOR};
            border-radius: 19px;
            min-height: 34px;
            font-size: 13px;
        }}
        QPushButton#loginBtn:hover, QPushButton#regBtn:hover {{ background-color: {BTN_HOVER}; }}
        QPushButton#loginBtn:pressed, QPushButton#regBtn:pressed {{ background-color: {INK_COLOR}; }}
    """)

    self.createGridLayout()
    self.setLayout(self.layout)
    self.show()

def make_label(self, text, svg):
    container = QWidget(self)
    row = QHBoxLayout(container)
    row.setContentsMargins(0, 0, 0, 0)
    row.setSpacing(6)

    text_lbl = QLabel(text, container)
    icon_lbl = QLabel(container)
    icon_lbl.setPixmap(svg_to_pixmap(svg, 24))
    icon_lbl.setFixedSize(QSize(24, 24))

    row.addWidget(text_lbl)
    row.addWidget(icon_lbl)
    row.addStretch()
    return container

def createGridLayout(self):
    self.layout = QGridLayout()
    self.layout.setContentsMargins(28, 16, 28, 24)
```

```
        self.layout.setVerticalSpacing(12)
        self.layout.setColumnStretch(0, 1)
        self.layout.setColumnStretch(1, 1)

        self.logo = LogoWidget(self)

        self.textbox = QLineEdit(self)
        self.password = QLineEdit(self)
        self.password.setEchoMode(QLineEdit.EchoMode.Password)
        self.textbox.returnPressed.connect(self.password.setFocus)
        self.password.returnPressed.connect(self.handle_login)

        self.login_button = QPushButton('Log In', self)
        self.login_button.setObjectName("loginBtn")
        self.login_button.setToolTip("Click to log in")
        self.login_button.setCursor(Qt.CursorShape.PointingHandCursor)
        self.login_button.clicked.connect(self.handle_login)

        self.register_button = QPushButton('Register', self)
        self.register_button.setObjectName("regBtn")
        self.register_button.setToolTip("Click to open registration form")
        self.register_button.setCursor(Qt.CursorShape.PointingHandCursor)
        self.register_button.clicked.connect(self.open_registration)

        self.layout.addWidget(self.logo, 0, 0, 1, 2, Qt.AlignmentFlag.AlignHCenter)
        self.layout.setRowMinimumHeight(1, 16)
        self.layout.addWidget(self.make_label("Username:", ICON_USERNAME), 2, 0)
        self.layout.addWidget(self.textbox, 2, 1)
        self.layout.addWidget(self.make_label("Password:", ICON_LOCK), 3, 0)
        self.layout.addWidget(self.password, 3, 1)
        self.layout.addWidget(self.login_button, 4, 1)
        self.layout.addWidget(self.register_button, 5, 1)
        self.layout.setRowStretch(6, 1)

    def handle_login(self):
        username = self.textbox.text().strip()
        password = self.password.text().strip()

        if not username or not password:
            self.shake()
            QMessageBox.warning(self, "Login Error", "Please enter both username and password.")
            return

        users = get_all_users()
        if not users:
            self.shake()
            QMessageBox.critical(self, "Error", "No registered accounts found. Please register first.")
            return
```

```
        user = get_user(username, password)

        if user and user[2] == password:
            self.shake()
            QMessageBox.information(self, "Menu Interface", f"Login Successful!\nWelcome to the Menu,
{username}!")
            self.textbox.clear()
            self.password.clear()

            self.open_chatroom(user, username)
            self.close()

        else:
            self.shake()
            QMessageBox.warning(self, "Login Failed", "Invalid username or password.")

    def open_registration(self):
        self.reg_window = RegistrationApp()
        self.reg_window.show()

    def open_chatroom(self, user, username):
        from MainChatroom import Main_Chatroom
        self.chat_window = Main_Chatroom(user_id=user[0], username=username)
        self.chat_window.show()

    def shake(self):
        if hasattr(self, "shake_anim") and self.shake_anim.state() == QPropertyAnimation.State.Running:
            return

        current_pos = self.pos()
        self.shake_anim = QPropertyAnimation(self, b"pos")
        self.shake_anim.setDuration(300)

        self.shake_anim.setKeyValueAt(0.00, current_pos)
        self.shake_anim.setKeyValueAt(0.20, current_pos + QPoint(-12, 0))
        self.shake_anim.setKeyValueAt(0.40, current_pos + QPoint(12, 0))
        self.shake_anim.setKeyValueAt(0.60, current_pos + QPoint(-8, 0))
        self.shake_anim.setKeyValueAt(0.80, current_pos + QPoint(8, 0))
        self.shake_anim.setKeyValueAt(1.00, current_pos)

        self.shake_anim.start()


if __name__ == '__main__':
    create_tables()

    try:
```

```
        import ctypes
        ctypes.windll.shell32.SetCurrentProcessExplicitAppUserModelID("dchat.app.1")
    except Exception:
        pass

    app = QApplication(sys.argv)
    app.setWindowIcon(make_app_icon())
    ex = App()
    sys.exit(app.exec())
```

Account Registration

Python

```
import sys
from PyQt6.QtCore import Qt, QByteArray, QSize
from PyQt6.QtWidgets import (QApplication, QWidget, QGridLayout, QHBoxLayout,
                             QLabel, QLineEdit, QPushButton, QMessageBox)
from PyQt6.QtGui import QPixmap, QPainter, QIcon
from PyQt6.QtSvg import QSvgRenderer

from database import create_user

BG_COLOR = "#F5F5DC"
INK_COLOR = "#000000"
BTN_COLOR = "#57534B"
BTN_HOVER = "#3F3C36"
BTN_TEXT = "#F5F5DC"

def _svg(body, color=INK_COLOR):
    return (f'<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" '
            f'fill="{color}" stroke="none">{body}</svg>')

ICON_USER = _svg('<circle cx="12" cy="7.5" r="5"/><path d="M2.5 22c0-5.5 4.2-8.5 9.5-8.5s9.5 3 9.5 8.5z"/>')

ICON_EMAIL = _svg(
    '<g fill="none" stroke="#000" stroke-width="1.8" stroke-linejoin="round">'
    '<rect x="2.5" y="4.5" width="19" height="15" rx="2"/>'
    '<path d="M3 5.5l9 7.5 9-7.5M3 19l6.5-7M21 19l-6.5-7"/></g>')

ICON_PHONE = _svg(
    '<path d="M6.62 10.79c1.44 2.83 3.76 5.14 6.59 6.59l2.2-2.2c.27-.27.67-.36 1.02-.24 '
    '1.12.37 2.33.57 3.57.57.55 0 1 .45 1 1V20c0 .55-.45 1-1 1-9.39 0-17-7.61-17-17 '
    '0-.55.45-1 1-1h3.5c.55 0 1 .45 1 1 0 1.25.2 2.45.57 3.57.11.35.03.74-.25 1.02l-2.2 2.2z"/>')

ICON_USERNAME = _svg(
```

```
    '<g fill="none" stroke="#000" stroke-width="1.8" stroke-linecap="round">'
    '<rect x="2.5" y="4" width="19" height="16" rx="3"/>'
    '<circle cx="8.5" cy="10" r="2"/><path d="M5.5 16c.5-2 2-2.8 3-2.8s2.5.8 3 2.8"/>'
    '<path d="M14.5 10h4M14.5 14h4"/></g>')

ICON_LOCK = _svg(
    '<path d="M18 8h-1V6c0-2.76-2.24-5-5-5S7 3.24 7 6v2H6c-1.1 0-2 .9-2 2v10c0 1.1.9 2 2 2h12'
    'c1.1 0 2-.9 2-2V10c0-1.1-.9-2-2-2zm-6 9c-1.1 0-2-.9-2-2s.9-2 2-2 2 .9 2 2-.9 2-2 2zm3.1-9'
    'H8.9V6c0-1.71 1.39-3.1 3.1-3.1 1.71 0 3.1 1.39 3.1 3.1v2z"/>')

APP_ICON_SVG = (
    '<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64">'
    f'<rect width="64" height="64" rx="14" fill="{BTN_COLOR}"/>'
    f'<g fill="{BG_COLOR}"><circle cx="26" cy="22" r="9"/>'
    '<path d="M8 50c0-10 8-15 18-15s18 5 18 15z"/>'
    f'<path d="M47 12h4v8h8v4h-8v8h-4v-8h-8v-4h8z" transform="translate(-6 -2)"/></g></svg>'
)


def svg_to_pixmap(svg_text, size):
    renderer = QSvgRenderer(QByteArray(svg_text.encode("utf-8")))
    pixmap = QPixmap(size, size)
    pixmap.fill(Qt.GlobalColor.transparent)
    painter = QPainter(pixmap)
    renderer.render(painter)
    painter.end()
    return pixmap


def make_app_icon():
    icon = QIcon()
    for s in (16, 32, 48, 64, 128, 256):
        icon.addPixmap(svg_to_pixmap(APP_ICON_SVG, s))
    return icon


class RegistrationApp(QWidget):
    def __init__(self):
        super().__init__()
        self.title = "Account Registration"
        self.initUI()

    def initUI(self):
        self.setWindowTitle(self.title)
        self.setWindowIcon(make_app_icon())
        self.setGeometry(200, 200, 560, 420)
        self.setMinimumSize(480, 380)
```

```
        self.setStyleSheet(f"""
            QWidget {{
                background-color: {BG_COLOR};
                color: {INK_COLOR};
                font-family: 'Segoe UI', sans-serif;
            }}
            QLabel {{ font-size: 16px; background: transparent; }}
            QLineEdit {{
                background-color: transparent;
                color: {INK_COLOR};
                border: 2px solid {INK_COLOR};
                border-radius: 19px;
                padding: 0 14px;
                font-size: 14px;
                min-width: 220px;
                min-height: 36px;
            }}
            QLineEdit:focus {{ background-color: #FFFFFF; }}
            QPushButton {{
                background-color: {BTN_COLOR};
                color: {BTN_TEXT};
                border: 2px solid {INK_COLOR};
                border-radius: 19px;
                min-height: 36px;
                font-size: 13px;
            }}
            QPushButton:hover {{ background-color: {BTN_HOVER}; }}
            QPushButton:pressed {{ background-color: {INK_COLOR}; }}
        """)

        self.createGridLayout()
        self.setLayout(self.layout)
        self.show()

    def make_label(self, text, svg):
        """Text label followed by an icon, like in the mockup."""
        container = QWidget(self)
        row = QHBoxLayout(container)
        row.setContentsMargins(0, 0, 0, 0)
        row.setSpacing(6)

        text_lbl = QLabel(text, container)
        icon_lbl = QLabel(container)
        icon_lbl.setPixmap(svg_to_pixmap(svg, 24))
        icon_lbl.setFixedSize(QSize(24, 24))

        row.addWidget(text_lbl)
        row.addWidget(icon_lbl)
```

```
        row.addStretch()
        return container

    def createGridLayout(self):
        self.layout = QGridLayout()
        self.layout.setContentsMargins(24, 24, 24, 24)
        self.layout.setVerticalSpacing(14)
        self.layout.setColumnStretch(0, 1)
        self.layout.setColumnStretch(1, 1)

        # Input widgets
        self.first_name = QLineEdit(self)
        self.last_name = QLineEdit(self)
        self.email = QLineEdit(self)
        self.contact = QLineEdit(self)
        self.textbox = QLineEdit(self)
        self.password = QLineEdit(self)
        self.password.setEchoMode(QLineEdit.EchoMode.Password)

        self.button = QPushButton("Register", self)
        self.button.setToolTip("Click to register your account")
        self.button.setCursor(Qt.CursorShape.PointingHandCursor)
        self.button.clicked.connect(self.register_account)

        rows = [
            ("First Name:", ICON_USER, self.first_name),
            ("Last Name:", ICON_USER, self.last_name),
            ("Email:", ICON_EMAIL, self.email),
            ("Contact Number:", ICON_PHONE, self.contact),
            ("Username:", ICON_USERNAME, self.textbox),
            ("Password:", ICON_LOCK, self.password),
        ]
        for i, (text, svg, field) in enumerate(rows):
            self.layout.addWidget(self.make_label(text, svg), i, 0)
            self.layout.addWidget(field, i, 1)

        self.layout.setRowMinimumHeight(len(rows), 10)
        self.layout.addWidget(self.button, len(rows) + 1, 1)

    def register_account(self):
        first_name = self.first_name.text().strip()
        last_name = self.last_name.text().strip()
        email = self.email.text().strip()
        contact = self.contact.text().strip()
        username = self.textbox.text().strip()
        password = self.password.text().strip()

        # Check for empty fields
```

```
        if not all([first_name, last_name, email, contact, username, password]):
            msg_box = QMessageBox(self)
            msg_box.setIcon(QMessageBox.Icon.Warning)
            msg_box.setWindowTitle("Registration Error")
            msg_box.setText("Please fill in all fields before registering.")
            msg_box.exec()
            return

        # Save the account to the database
        success, msg = create_user(username, password)

        if success:
            msg_box = QMessageBox(self)
            msg_box.setIcon(QMessageBox.Icon.Information)
            msg_box.setWindowTitle("Registration Successful")
            msg_box.setText(f"Account for '{username}' registered successfully!")
            msg_box.exec()

            # Clear inputs
            self.first_name.clear()
            self.last_name.clear()
            self.email.clear()
            self.contact.clear()
            self.textbox.clear()
            self.password.clear()
            self.close()
        else:
            msg_box = QMessageBox(self)
            msg_box.setIcon(QMessageBox.Icon.Critical)
            msg_box.setWindowTitle("Registration Error")
            msg_box.setText(msg)
            msg_box.exec()


if __name__ == '__main__':
    try:
        import ctypes
        ctypes.windll.shell32.SetCurrentProcessExplicitAppUserModelID("registration.app.1")
    except Exception:
        pass

    app = QApplication(sys.argv)
    app.setWindowIcon(make_app_icon())
    ex = RegistrationApp()

    sys.exit(app.exec())
```

Main Chatroom System

Python

```
import sys
import os
from PyQt6.QtCore import Qt, QByteArray, QSize
from PyQt6.QtWidgets import (QApplication, QWidget, QGridLayout, QLabel, QLineEdit, QPushButton,
QMessageBox,
                             QMainWindow, QVBoxLayout, QHBoxLayout, QListWidget, QListWidgetItem, QDialog,
                             QFileDialog, QFrame)
from PyQt6.QtGui import QPixmap, QPainter, QPainterPath, QIcon
from PyQt6.QtSvg import QSvgRenderer

from database import (save_message, get_all_users, get_messages,
                      update_user_avatar, update_username, get_user_by_id)

from MenuInterface import App

# ---------------------------------------------------------- Palette
BG_COLOR = "#F5F5DC"
PANEL_COLOR = "#CBC7B6"
PANEL_SELECT = "#B8B4A1"
INK_COLOR = "#000000"
BTN_COLOR = "#57534B"
BTN_HOVER = "#3F3C36"
BTN_TEXT = "#F5F5DC"
MUTED_TEXT = "#57534B"

# ------------------------------------------------------------
ICON_USER = ('<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24" fill="#000">'
             '<circle cx="12" cy="7.5" r="5"/><path d="M2.5 22c0-5.5 4.2-8.5 9.5-8.5s9.5 3 9.5 8.5z"/></svg>')

ICON_SMILEY = ('<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24">'
               '<circle cx="12" cy="12" r="10" fill="#FFD21F" stroke="#8A6A00" stroke-width="1.2"/>'
               '<ellipse cx="8.8" cy="9.5" rx="1" ry="1.6" fill="#000"/>'
               '<ellipse cx="15.2" cy="9.5" rx="1" ry="1.6" fill="#000"/>'
               '<path d="M6.8 14c1.3 2.8 9.1 2.8 10.4 0" fill="none" stroke="#000" '
               'stroke-width="1.4" stroke-linecap="round"/></svg>')

APP_ICON_SVG = (
    '<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64">'
    f'<rect width="64" height="64" rx="14" fill="{BTN_COLOR}"/>'
    f'<rect x="10" y="13" width="44" height="30" rx="11" fill="{BG_COLOR}"/>'
    f'<path d="M20 40l0 12 13-12z" fill="{BG_COLOR}"/>'
    f'<g fill="{BTN_COLOR}"><circle cx="22" cy="28" r="3"/><circle cx="32" cy="28" r="3"/>'
    '<circle cx="42" cy="28" r="3"/></g></svg>'
```

```
)


def svg_to_pixmap(svg_text, size):
    renderer = QSvgRenderer(QByteArray(svg_text.encode("utf-8")))
    pixmap = QPixmap(size, size)
    pixmap.fill(Qt.GlobalColor.transparent)
    painter = QPainter(pixmap)
    renderer.render(painter)
    painter.end()
    return pixmap


def make_app_icon():
    icon = QIcon()
    for s in (16, 32, 48, 64, 128, 256):
        icon.addPixmap(svg_to_pixmap(APP_ICON_SVG, s))
    return icon


def make_transparent(list_widget):
    """Lets the rounded panel colour show through the list's viewport."""
    list_widget.viewport().setAutoFillBackground(False)
    list_widget.viewport().setStyleSheet("background: transparent;")

STYLE = f"""
    QMainWindow, QWidget, QDialog {{
        background-color: {BG_COLOR};
        color: {INK_COLOR};
        font-family: 'Segoe UI', sans-serif;
    }}
    QLabel {{ background: transparent; color: {INK_COLOR}; }}

    /* Rounded taupe panels */
    QListWidget#contactsWidget {{
        background-color: {PANEL_COLOR};
        border: 2px solid {INK_COLOR};
        border-radius: 34px;
        padding: 14px 10px;
        font-size: 14px;
        outline: none;
    }}
    QListWidget#contactsWidget::item {{ border-radius: 14px; margin-bottom: 2px; }}
    QListWidget#contactsWidget::item:hover {{ background-color: {PANEL_SELECT}; }}
    QListWidget#contactsWidget::item:selected {{ background-color: {PANEL_SELECT}; }}

    QFrame#chatPanel {{
        background-color: {PANEL_COLOR};
```

```
    border: 2px solid {INK_COLOR};
    border-radius: 34px;
}}
QListWidget#chatWidget {{
    background: transparent;
    border: none;
    outline: none;
}}
QListWidget#chatWidget::item {{ border: none; background: transparent; }}
QListWidget#chatWidget::item:selected {{ background: transparent; }}

QScrollBar:vertical {{ background: transparent; width: 10px; margin: 14px 2px; }}
QScrollBar::handle:vertical {{ background: {BTN_COLOR}; border-radius: 3px; min-height: 24px; }}
QScrollBar::add-line:vertical, QScrollBar::sub-line:vertical {{ height: 0; }}
QScrollBar::add-page:vertical, QScrollBar::sub-page:vertical {{ background: transparent; }}

/* Pill inputs */
QLineEdit {{
    background-color: {BG_COLOR};
    color: {INK_COLOR};
    border: 2px solid {INK_COLOR};
    border-radius: 19px;
    padding: 0 16px;
    font-size: 13px;
    min-height: 34px;
}}
QLineEdit:focus {{ background-color: #FFFFFF; }}

/* Pill buttons: outline style (Send, Emoji, Profile) */
QPushButton#sendBtn, QPushButton#emojiBtn, QPushButton#profileBtn, QPushButton#outlineBtn {{
    background-color: {BG_COLOR};
    color: {INK_COLOR};
    border: 2px solid {INK_COLOR};
    border-radius: 19px;
    min-height: 34px;
    font-size: 13px;
}}
QPushButton#sendBtn {{ padding: 0 20px; }}
QPushButton#profileBtn {{ padding: 2px 16px; font-size: 12px; border-radius: 20px; }}
QPushButton#sendBtn:hover, QPushButton#emojiBtn:hover,
QPushButton#profileBtn:hover, QPushButton#outlineBtn:hover {{ background-color: {PANEL_COLOR}; }}

/* Pill buttons: solid dark style (dialog primary actions) */
QPushButton#primaryBtn {{
    background-color: {BTN_COLOR};
    color: {BTN_TEXT};
    border: 2px solid {INK_COLOR};
    border-radius: 19px;
```

```
        min-height: 34px;
        font-size: 13px;
        font-weight: bold;
    }}
    QPushButton#primaryBtn:hover {{ background-color: {BTN_HOVER}; }}
    QPushButton#primaryBtn:pressed {{ background-color: {INK_COLOR}; }}
"""

# Helper function to crop and render smooth circular pixmaps for avatars
def get_circular_pixmap(image_path, size=28):
    if image_path and os.path.exists(image_path):
        src_pixmap = QPixmap(image_path)
        if not src_pixmap.isNull():
            scaled = src_pixmap.scaled(size, size, Qt.AspectRatioMode.KeepAspectRatioByExpanding,
Qt.TransformationMode.SmoothTransformation)
            out_pixmap = QPixmap(size, size)
            out_pixmap.fill(Qt.GlobalColor.transparent)

            painter = QPainter(out_pixmap)
            painter.setRenderHint(QPainter.RenderHint.Antialiasing)
            path = QPainterPath()
            path.addEllipse(0, 0, size, size)
            painter.setClipPath(path)
            painter.drawPixmap(0, 0, scaled)
            painter.end()
            return out_pixmap
    return None

# Profile Edit Dialog
class ProfileDialog(QDialog):
    def __init__(self, user_id, current_username, current_avatar, parent=None):
        super().__init__(parent)
        self.user_id = user_id
        self.current_username = current_username
        self.current_avatar = current_avatar
        self.new_avatar_path = current_avatar

        self.setWindowTitle("Edit Profile")
        self.setWindowIcon(make_app_icon())
        self.setStyleSheet(STYLE)
        self.setFixedSize(320, 360)
        self.initUI()

    def initUI(self):
        layout = QVBoxLayout(self)
        layout.setContentsMargins(24, 20, 24, 20)
        layout.setSpacing(12)
```

```
        self.avatar_preview = QLabel()
        self.avatar_preview.setFixedSize(64, 64)
        self.avatar_preview.setAlignment(Qt.AlignmentFlag.AlignCenter)
        self.update_avatar_preview(self.current_avatar)

        avatar_container = QHBoxLayout()
        avatar_container.addStretch()
        avatar_container.addWidget(self.avatar_preview)
        avatar_container.addStretch()
        layout.addLayout(avatar_container)
        layout.addSpacing(10)

        change_avatar_btn = QPushButton("Change Avatar")
        change_avatar_btn.setObjectName("outlineBtn")
        change_avatar_btn.setCursor(Qt.CursorShape.PointingHandCursor)
        change_avatar_btn.clicked.connect(self.select_new_avatar)

        btn_container = QHBoxLayout()
        btn_container.addStretch()
        btn_container.addWidget(change_avatar_btn)
        btn_container.addStretch()
        layout.addLayout(btn_container)

        # Username Input
        user_label = QLabel("Username")
        user_label.setStyleSheet("font-size: 13px; font-weight: bold; margin-top: 4px;")
        layout.addWidget(user_label)

        self.username_input = QLineEdit(self.current_username)
        layout.addWidget(self.username_input)

        layout.addStretch()

        # Save Button
        save_btn = QPushButton("Save Changes")
        save_btn.setObjectName("primaryBtn")
        save_btn.setCursor(Qt.CursorShape.PointingHandCursor)
        save_btn.clicked.connect(self.save_profile)
        layout.addWidget(save_btn)

        # Logout Button
        logout_btn = QPushButton("Log Out")
        logout_btn.setObjectName("primaryBtn")
        logout_btn.setCursor(Qt.CursorShape.PointingHandCursor)
        logout_btn.clicked.connect(self.logout)
        layout.addWidget(logout_btn)

    def update_avatar_preview(self, path):
```

```
        pix = get_circular_pixmap(path, size=64)
        if pix:
            self.avatar_preview.setPixmap(pix)
            self.avatar_preview.setStyleSheet("border: none; background: transparent;")
        else:
            initial = self.current_username[0].upper() if self.current_username else "U"
            self.avatar_preview.setText(initial)
            self.avatar_preview.setStyleSheet(f"""
                background-color: {BTN_COLOR};
                color: {BTN_TEXT};
                border: 2px solid {INK_COLOR};
                border-radius: 32px;
                font-size: 24px;
                font-weight: bold;
            """)

    def select_new_avatar(self):
        file_path, _ = QFileDialog.getOpenFileName(
            self, "Select Avatar Image", "", "Image Files (*.png *.jpg *.jpeg *.bmp)"
        )
        if file_path:
            self.new_avatar_path = file_path
            self.update_avatar_preview(file_path)

    def save_profile(self):
        new_name = self.username_input.text().strip()
        if not new_name:
            QMessageBox.warning(self, "Error", "Username cannot be empty.")
            return

        if new_name != self.current_username:
            success, msg = update_username(self.user_id, new_name)
            if not success:
                QMessageBox.warning(self, "Error", msg)
                return

        if self.new_avatar_path != self.current_avatar:
            update_user_avatar(self.user_id, self.new_avatar_path)

        QMessageBox.information(self, "Success", "Profile updated successfully!")
        self.accept()

    def logout(self):
        reply = QMessageBox.warning(
            self, "Log Out", "Are you sure you want to log out?",
            QMessageBox.StandardButton.Yes | QMessageBox.StandardButton.No,
        )
        if reply == QMessageBox.StandardButton.Yes:
```

```
            self.close()
            self.parent().logout()

class EmojiPicker(QDialog):
    def __init__(self, parent=None):
        super().__init__(parent)
        self.setWindowTitle("Select Emoji")
        self.setWindowIcon(make_app_icon())
        self.setStyleSheet(STYLE)
        self.setFixedSize(230, 180)
        self.selected_emoji = ""

        grid = QGridLayout()
        grid.setContentsMargins(12, 12, 12, 12)
        emojis = [
            '😊', '😂', '👍', '❤️', '🎉',
            '🔥', '😎', '🙏', '😭', '✨',
            '👋', '🤔', '💯', '🚀', '🥳'
        ]

        positions = [(i, j) for i in range(3) for j in range(5)]
        for pos, emoji in zip(positions, emojis):
            btn = QPushButton(emoji)
            btn.setFixedSize(36, 36)
            btn.setStyleSheet(f"""
                QPushButton {{ font-size: 16px; border: 1.5px solid {INK_COLOR}; border-radius: 18px; background:
{BG_COLOR}; color: {INK_COLOR}; }}
                QPushButton:hover {{ background: {PANEL_COLOR}; }}
            """)
            btn.clicked.connect(lambda _, e=emoji: self.pick_emoji(e))
            grid.addWidget(btn, *pos)

        self.setLayout(grid)

    def pick_emoji(self, emoji):
        self.selected_emoji = emoji
        self.accept()

class ContactItemWidget(QWidget):
    def __init__(self, username, avatar_path):
        super().__init__()
        self.setStyleSheet("background: transparent;")

        layout = QHBoxLayout(self)
        layout.setContentsMargins(4, 4, 4, 4)
        layout.setSpacing(10)

        avatar_label = QLabel()
```

```
        avatar_label.setFixedSize(28, 28)
        avatar_label.setAlignment(Qt.AlignmentFlag.AlignCenter)

        circular_pix = get_circular_pixmap(avatar_path, size=28)
        if circular_pix:
            avatar_label.setPixmap(circular_pix)
        else:
            initial = username[0].upper() if username else "U"
            avatar_label.setText(initial)
            avatar_label.setStyleSheet(f"""
                background-color: {BTN_COLOR};
                color: {BTN_TEXT};
                border-radius: 14px;
                font-size: 12px;
                font-weight: bold;
            """)

        name_label = QLabel(username)
        name_label.setStyleSheet(f"font-size: 13px; font-weight: 600; color: {INK_COLOR}; background:
transparent;")

        layout.addWidget(avatar_label)
        layout.addWidget(name_label)
        layout.addStretch()

class MessageBubble(QWidget):
    def __init__(self, message, timestamp, is_self):
        super().__init__()
        self.setStyleSheet("background: transparent;")

        main_layout = QVBoxLayout(self)
        main_layout.setContentsMargins(8, 4, 8, 4)
        main_layout.setSpacing(2)

        container = QWidget()
        box_layout = QVBoxLayout(container)
        box_layout.setContentsMargins(14, 8, 14, 8)

        msg_label = QLabel(message)
        msg_label.setWordWrap(True)

        time_label = QLabel(timestamp)
        time_label.setStyleSheet(f"font-size: 9px; font-family: 'Segoe UI', sans-serif; background: transparent; border:
none; color: {MUTED_TEXT}; margin-top: 1px;")

        bubble_row = QHBoxLayout()

        if is_self:
```

```
            container.setStyleSheet(f"QWidget {{ background-color: {BTN_COLOR}; border: 2px solid {INK_COLOR};
border-radius: 14px; }}")
            msg_label.setStyleSheet(f"font-size: 13px; font-family: 'Segoe UI', sans-serif; background: transparent;
border: none; color: {BTN_TEXT};")
            box_layout.addWidget(msg_label)

            bubble_row.addStretch()
            bubble_row.addWidget(container)

            main_layout.addLayout(bubble_row)
            main_layout.addWidget(time_label, alignment=Qt.AlignmentFlag.AlignRight)
        else:
            container.setStyleSheet(f"QWidget {{ background-color: {BG_COLOR}; border: 2px solid {INK_COLOR};
border-radius: 14px; }}")
            msg_label.setStyleSheet(f"font-size: 13px; font-family: 'Segoe UI', sans-serif; background: transparent;
border: none; color: {INK_COLOR};")
            box_layout.addWidget(msg_label)

            bubble_row.addWidget(container)
            bubble_row.addStretch()

            main_layout.addLayout(bubble_row)
            main_layout.addWidget(time_label, alignment=Qt.AlignmentFlag.AlignLeft)


class Main_Chatroom(QMainWindow):
    def __init__(self, user_id, username='Guest'):
        super().__init__()
        self.username = username
        self.user_id = user_id
        self.receiver_id = None
        self.user_avatar = ""

        user_info = get_user_by_id(self.user_id)
        if user_info:
            self.username = user_info[1]
            self.user_avatar = user_info[2] or ""

        self.title = f"Local Chatroom - Logged in as: {self.username}"
        self.x = 200
        self.y = 200
        self.width = 800
        self.height = 500
        self.initUI()
        self.load_contacts()

    def initUI(self):
        self.setWindowTitle(self.title)
```

```
self.setWindowIcon(make_app_icon())
self.setGeometry(self.x, self.y, self.width, self.height)
self.setStyleSheet(STYLE)

central_widget = QWidget()
self.setCentralWidget(central_widget)

main_layout = QVBoxLayout(central_widget)
main_layout.setContentsMargins(16, 14, 16, 16)
main_layout.setSpacing(10)

# Header Bar
top_bar = QHBoxLayout()
app_title = QLabel("Local Chatroom")
app_title.setStyleSheet("font-size: 20px; font-weight: bold;")

self.profile_btn = QPushButton()
self.profile_btn.setObjectName("profileBtn")
self.profile_btn.setToolTip("Edit Profile")
self.profile_btn.setCursor(Qt.CursorShape.PointingHandCursor)
self.profile_btn.clicked.connect(self.open_profile_dialog)
self.refresh_profile_header()

top_bar.addWidget(app_title)
top_bar.addStretch()
top_bar.addWidget(self.profile_btn)

main_layout.addLayout(top_bar)

content_layout = QHBoxLayout()
content_layout.setSpacing(16)

self.user_chats = QVBoxLayout()
self.user_chats.setContentsMargins(0, 0, 0, 0)
self.chats_display = QLabel("Chats")
self.chats_display.setStyleSheet("font-size: 20px; margin-left: 8px;")
self.user_chats.addWidget(self.chats_display)

self.contacts = QListWidget()
self.contacts.setObjectName("contactsWidget")
self.contacts.itemClicked.connect(self.select_contacts)
make_transparent(self.contacts)
self.user_chats.addWidget(self.contacts)

contacts_list = QWidget()
contacts_list.setLayout(self.user_chats)
contacts_list.setMaximumWidth(210)
content_layout.addWidget(contacts_list)
```

```
        chat_panel = QFrame()
        chat_panel.setObjectName("chatPanel")
        chat_area = QVBoxLayout(chat_panel)
        chat_area.setContentsMargins(18, 18, 18, 16)
        chat_area.setSpacing(10)

        self.chat_display = QListWidget()
        self.chat_display.setObjectName("chatWidget")
        self.chat_display.setSelectionMode(QListWidget.SelectionMode.NoSelection)
        make_transparent(self.chat_display)
        chat_area.addWidget(self.chat_display)

        send_text = QHBoxLayout()
        send_text.setSpacing(8)

        self.text_input = QLineEdit()
        self.text_input.setPlaceholderText("Type a message (max 300 characters)...")
        self.text_input.setMaxLength(300)
        self.text_input.returnPressed.connect(self.send_message)
        send_text.addWidget(self.text_input)

        self.emojiButton = QPushButton()
        self.emojiButton.setObjectName("emojiBtn")
        self.emojiButton.setIcon(QIcon(svg_to_pixmap(ICON_SMILEY, 24)))
        self.emojiButton.setIconSize(QSize(22, 22))
        self.emojiButton.setFixedSize(52, 38)
        self.emojiButton.setCursor(Qt.CursorShape.PointingHandCursor)
        self.emojiButton.setToolTip("Insert emoji")
        self.emojiButton.clicked.connect(self.open_emoji_picker)
        send_text.addWidget(self.emojiButton)

        self.sendButton = QPushButton("Send")
        self.sendButton.setObjectName("sendBtn")
        self.sendButton.setFixedHeight(38)
        self.sendButton.setCursor(Qt.CursorShape.PointingHandCursor)
        self.sendButton.clicked.connect(self.send_message)
        send_text.addWidget(self.sendButton)

        chat_area.addLayout(send_text)

        content_layout.addWidget(chat_panel, 1)

        main_layout.addLayout(content_layout)
        self.show()

    def refresh_profile_header(self):
        user_info = get_user_by_id(self.user_id)
```

```
        if user_info:
            self.username = user_info[1]
            self.user_avatar = user_info[2] or ""

        pix = get_circular_pixmap(self.user_avatar, size=24)
        if pix:
            self.profile_btn.setIcon(QIcon(pix))
        else:
            self.profile_btn.setIcon(QIcon(svg_to_pixmap(ICON_USER, 24)))
        self.profile_btn.setIconSize(QSize(24, 24))
        self.profile_btn.setText(f"  {self.username}")

        self.setWindowTitle(f"Chatroom - Logged in as: {self.username}")

    def open_profile_dialog(self):
        dialog = ProfileDialog(self.user_id, self.username, self.user_avatar, self)
        if dialog.exec():
            self.refresh_profile_header()
            self.load_contacts()

    def open_emoji_picker(self):
        picker = EmojiPicker(self)
        if picker.exec() and picker.selected_emoji:
            self.text_input.setText(self.text_input.text() + picker.selected_emoji)
            self.text_input.setFocus()

    def load_contacts(self):
        self.contacts.clear()

        for user_data in get_all_users():
            user_id = user_data[0]
            username = user_data[1]
            avatar_path = user_data[2] if len(user_data) > 2 else ""

            if int(user_id) == self.user_id:
                continue

            contact_widget = ContactItemWidget(username, avatar_path)
            item = QListWidgetItem(self.contacts)
            item.setData(Qt.ItemDataRole.UserRole, int(user_id))
            item.setText("")
            item.setSizeHint(contact_widget.sizeHint())

            self.contacts.addItem(item)
            self.contacts.setItemWidget(item, contact_widget)

    def select_contacts(self, item):
        self.receiver_id = int(item.data(Qt.ItemDataRole.UserRole))
```

```
        self.chat_display.clear()

        for msg in get_messages(self.user_id, self.receiver_id):
            message_id, sender_id, receiver_id, message, timestamp = msg
            is_self = (int(sender_id) == self.user_id)

            bubble_widget = MessageBubble(message, timestamp, is_self)
            list_item = QListWidgetItem(self.chat_display)
            list_item.setSizeHint(bubble_widget.sizeHint())

            self.chat_display.addItem(list_item)
            self.chat_display.setItemWidget(list_item, bubble_widget)

        self.chat_display.scrollToBottom()

    def send_message(self):
        message = self.text_input.text().strip()

        if not message:
            return

        if self.receiver_id is None:
            QMessageBox.warning(self, "No Recipient", "Please Select a contact first.")
            return

        save_message(self.user_id, self.receiver_id, message)
        self.text_input.clear()

        if self.contacts.currentItem():
            self.select_contacts(self.contacts.currentItem())

    def logout(self):
        self.menu = App()
        self.menu.show()
        self.close()


if __name__ == '__main__':
    try:
        import ctypes
        ctypes.windll.shell32.SetCurrentProcessExplicitAppUserModelID("local.chatroom.app.1")
    except Exception:
        pass

    app = QApplication(sys.argv)
    app.setWindowIcon(make_app_icon())
    ex = Main_Chatroom(user_id=1, username='TestUser')
```

```
sys.exit(app.exec())
```

Outputs

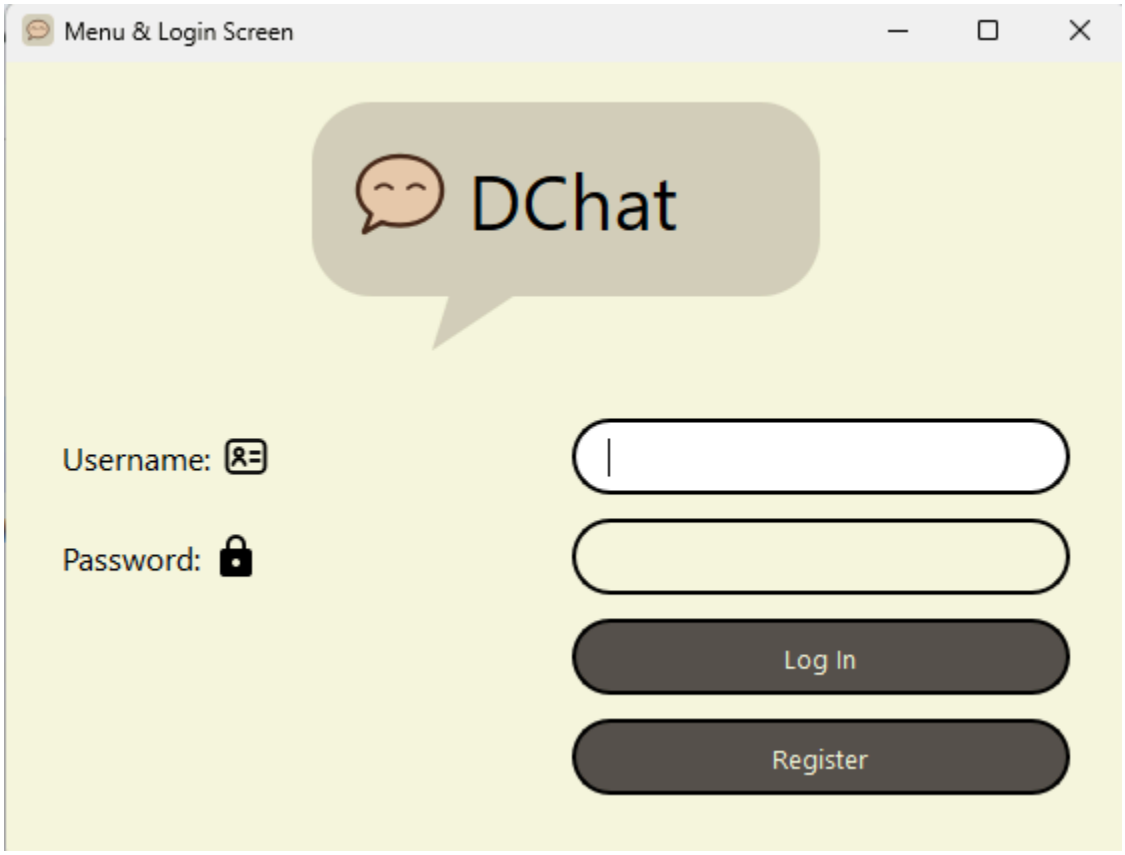


Figure 1: Menu & Login Screen

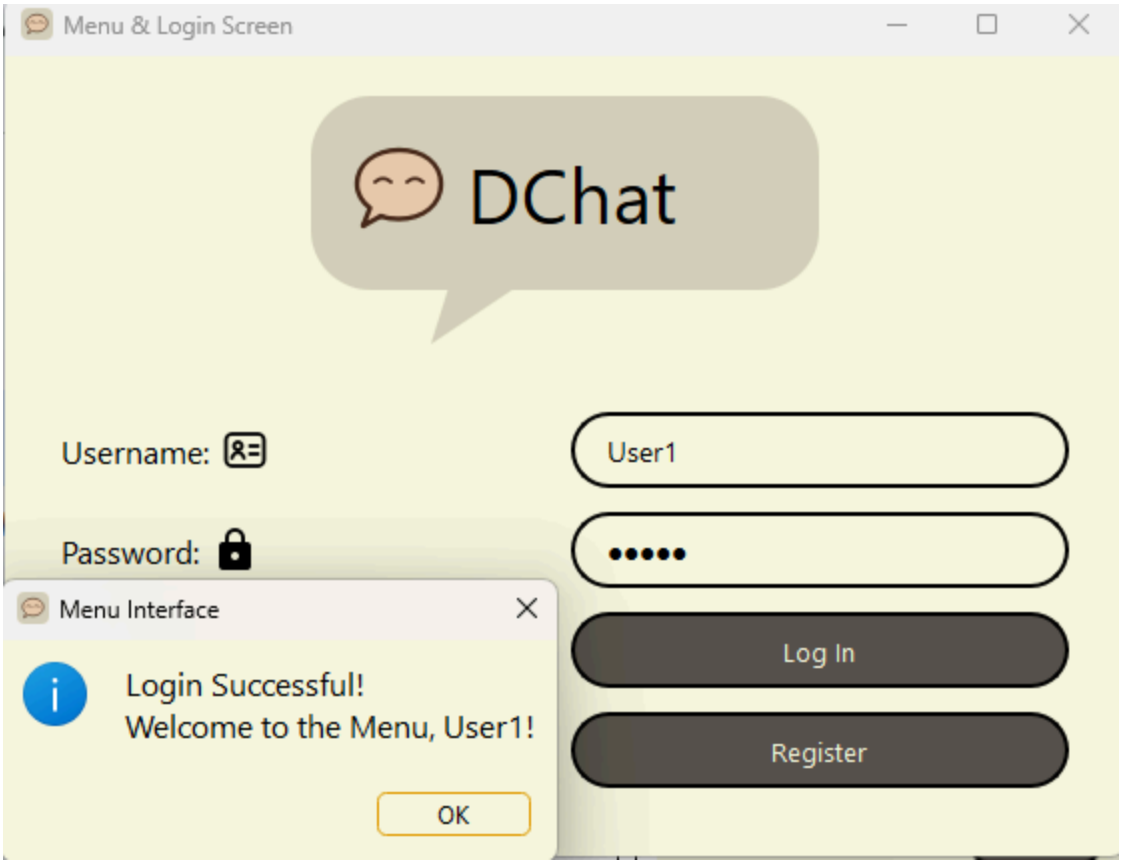


Figure 2: Login Successful Popup

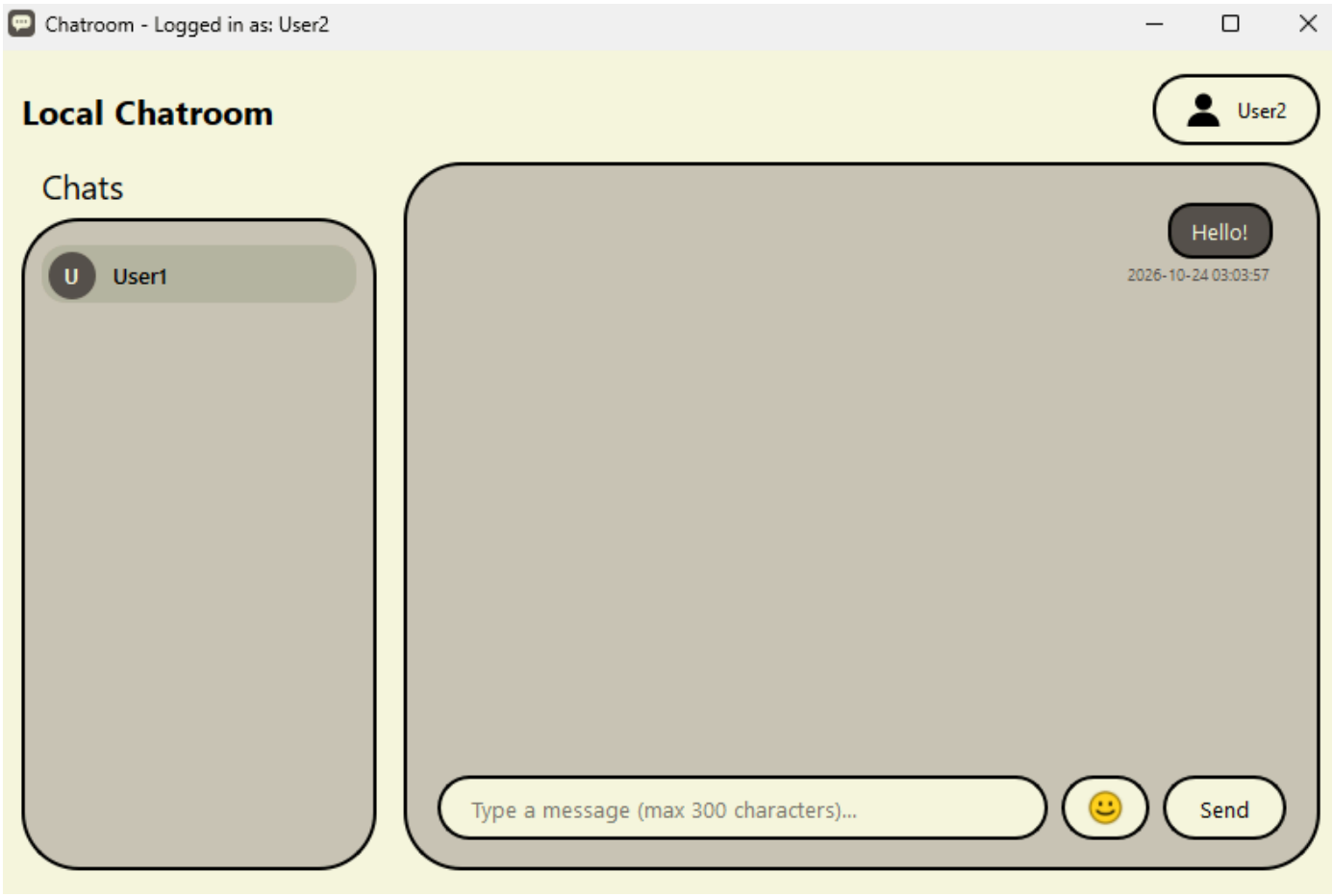


Figure 3: Main Chatroom as User2

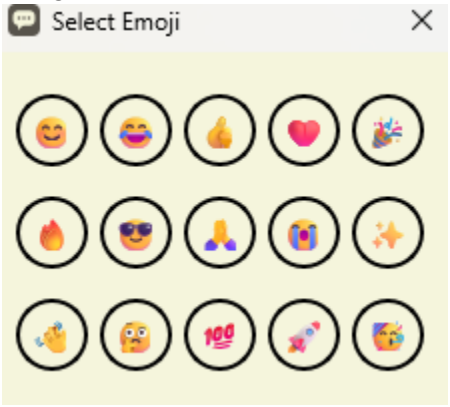


Figure 4: Selection of Emojis

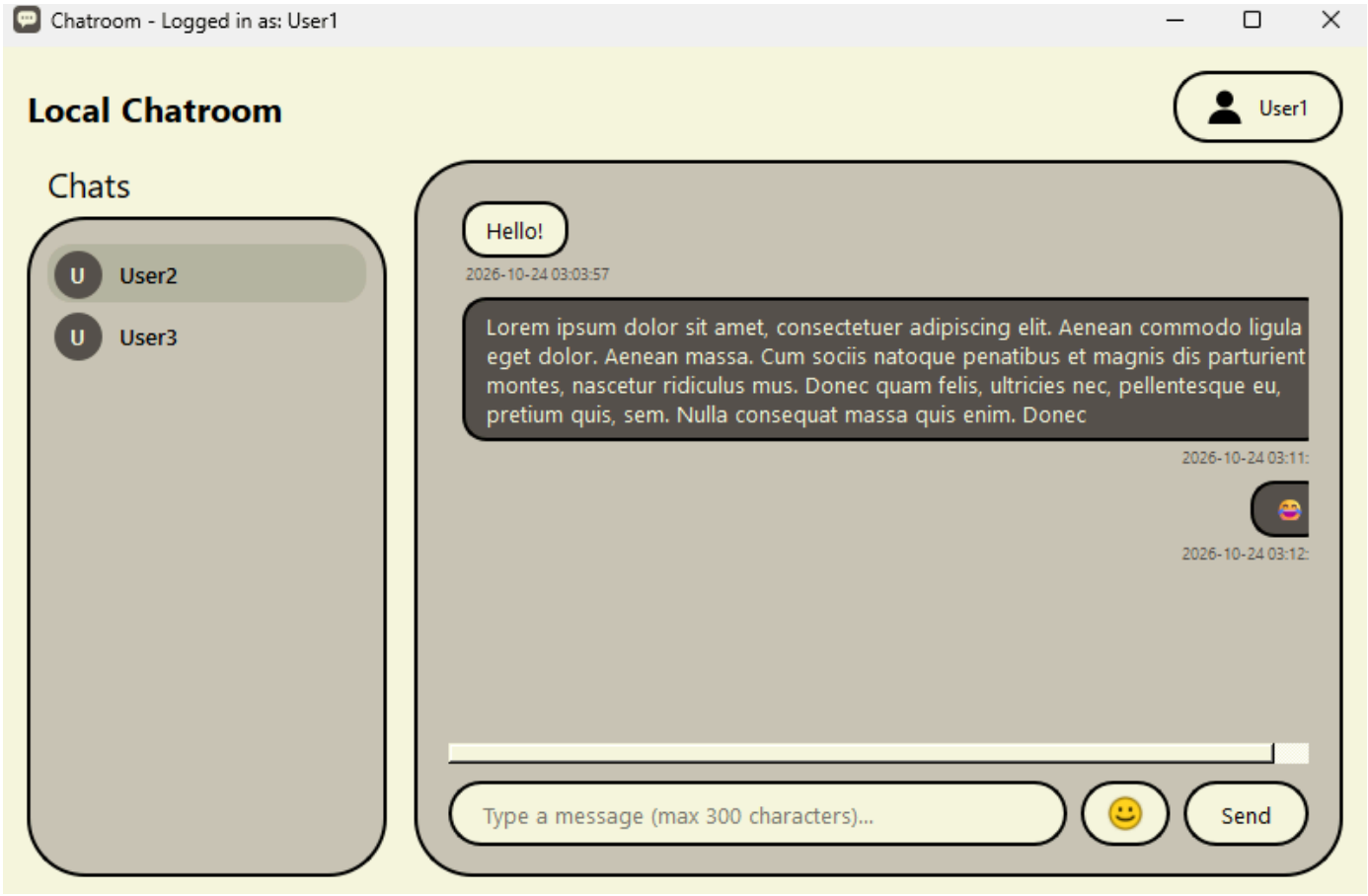


Figure 5: Sample Chat Message and Emojis with Maximum Limit of Characters

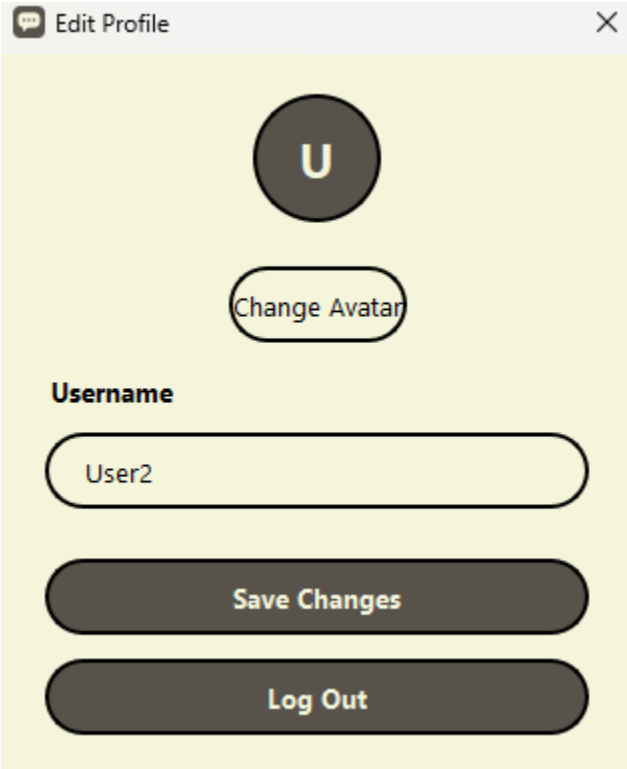


Figure 6: Editing of Profile & Change of Avatar

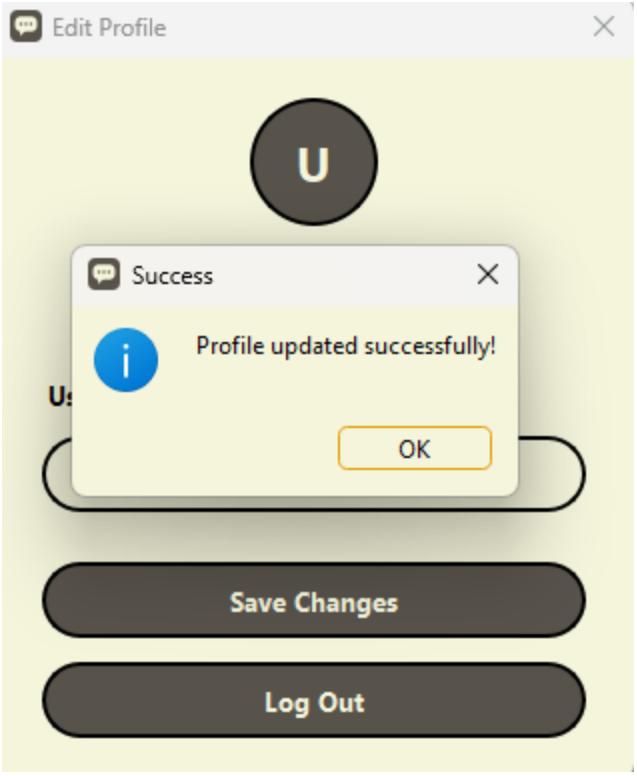


Figure 7: Saved Changes of Avatar

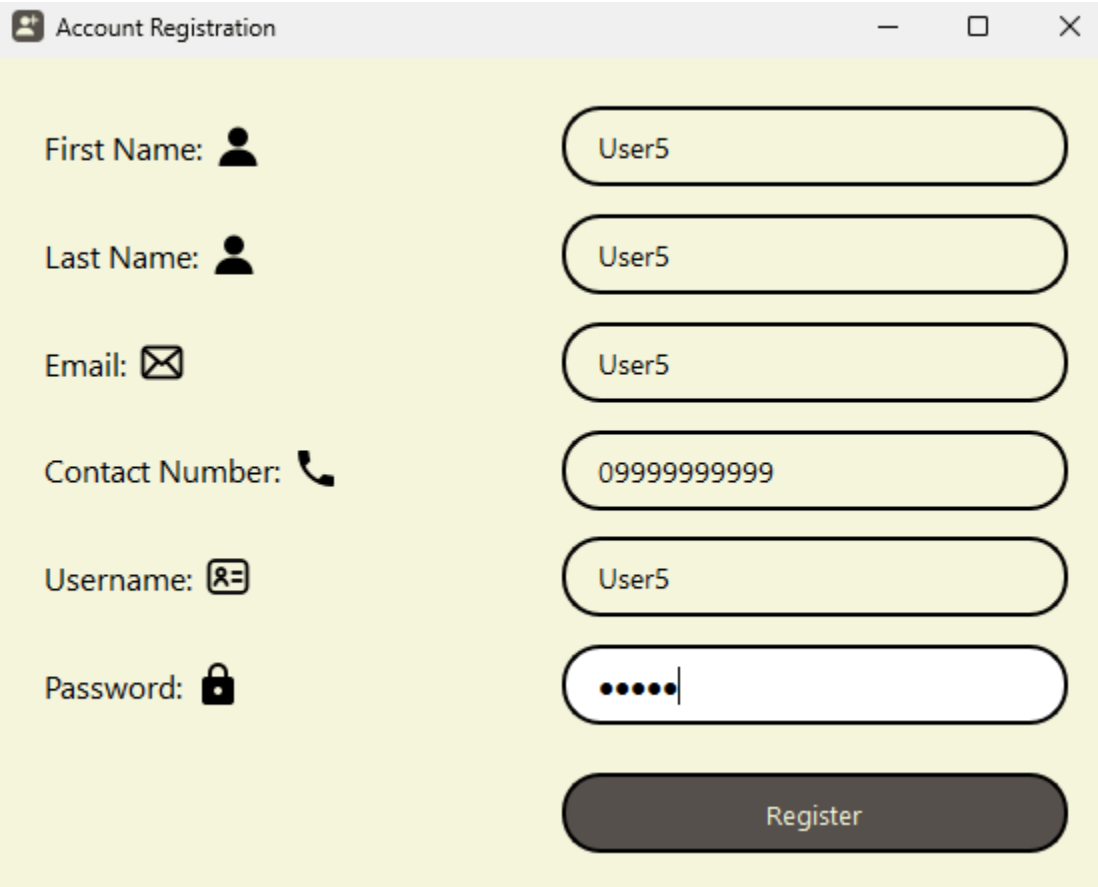


Figure 8: Account Registration

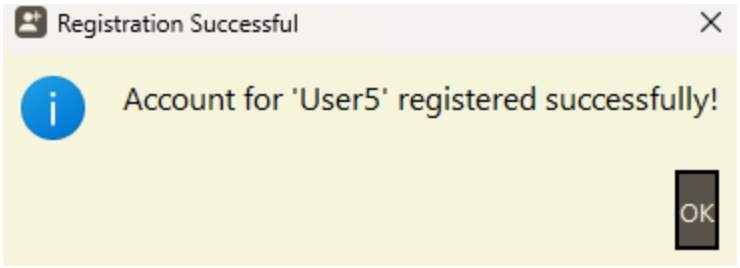


Figure 9: Successful Registration Popup



Figure 10: Main Chatroom as User2

B. Output(s) and Observation(s)

GitHub Repository
https://github.com/jplara-cpe2025/CPE-009B-CPE21S2-FINAL-PROJECT-GROUP2-2526

C. Conclusion & Lessons Learned